Anchor-Guided Dequantization
for Federated Learning

Repairing low-bit client updates with full-precision anchors

Jacob Simmons (jss0112@auburn.edu)
Owen Strength (ods0005@auburn.edu)
Auburn University
COMP 5970/6970 Course Project

CIFAR-10 10 clients INT2 / INT4 / INT8 server-side CorrectionNet

Problem Setting

Standard FL assumption

FedAvg works well when uploaded client updates are already comparable and can be averaged directly.

What changes here

Clients are **bit-width heterogeneous**.

- Anchor clients upload **FP32** deltas.
- Constrained clients upload **INT8 / INT4 / INT2** deltas.
- The server averages *clean* and *biased* updates together.

Failure mode

Quantization error is not just random noise.

- Low-bit updates lose magnitude information.
- With very few levels, signs cancel and the average collapses.
- The global model absorbs that distortion every round.

Bias at the server

If the constrained upload is a distorted version of the full-precision update, then averaging clean and quantized deltas pushes the server in the wrong direction.

Why Existing Fixes Are Not Enough

FedAvg

- No correction step at all.
- Assumes averaging cancels quantization noise; breaks hardest at INT2.

FedShift

- One global anchor-based shift per round.
- Strong near-IID baseline, but still one target for everyone.

ProWD

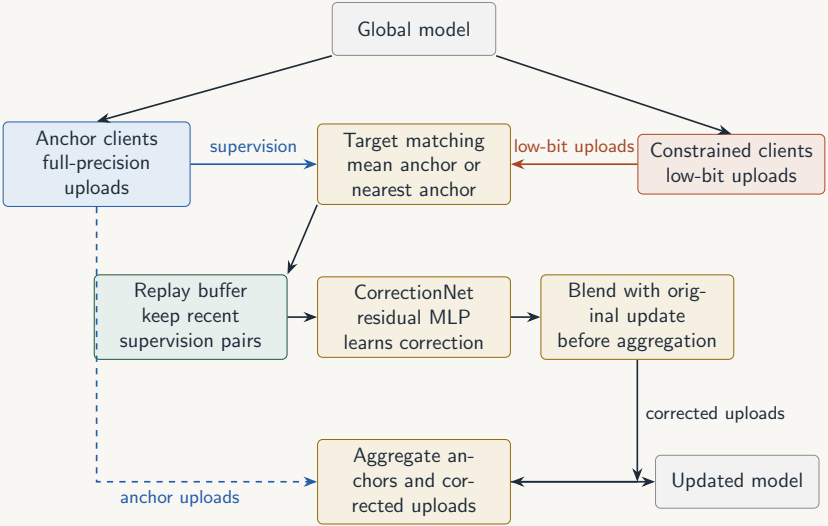
- Learned residual MLP, but trained on the current round only.
- Pulls every constrained client toward the same mean anchor target.

AGD-Smart

- Replay buffer plus direct anchor supervision.
- Client-specific matching instead of one global target.
- Blended correction when non-IID structure matters.

AGD-Smart is the only variant that is both persistent and client-specific.

Method Overview



What Happens Each Round

1. **Collect two kinds of updates**
   - Anchors send clean FP32 deltas Δw_a^{fp}.
   - Constrained clients send quantized deltas Δw_j^q.
2. **Build targets for constrained clients**
   - Baseline mode: target the *mean* anchor delta.
   - Smart mode: choose the anchor whose update points in the most similar direction.
3. **Train on buffered supervision**
   - Store $(\Delta w_j^q, \Delta w_j^{\mathrm{target}})$ pairs.
   - Keep the most recent **10 rounds**.
   - Normalize buffered inputs with a scalar z-score before training.
4. **Correct, blend, aggregate**
   - Apply the learned map to each constrained delta.
   - Blend corrected and original directions when needed.
   - Aggregate anchors plus corrected constrained updates.

CorrectionNet and Training Objective

CorrectionNet

- One encoder layer
- Three residual blocks
- Zero-initialized decoder

Zero-init matters: training starts from an identity map, so the network learns *bias correction* instead of a full rewrite of the update.

Loss

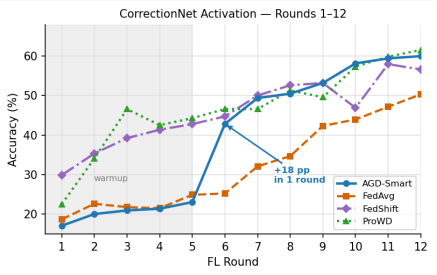
- Reconstruction term: make the corrected upload look like the chosen anchor target.
- Optional distillation term: make the repaired update produce similar logits on a server validation split.
- $\tau = 4$ in the reported settings.

Why Warmup Matters

Warmup

For the first **5 rounds**, the server trains CorrectionNet but does not apply correction yet.

- The replay buffer fills with real supervision pairs.
- The untrained network cannot damage early optimization.
- Correction activates only after there is enough signal to use.



Interpretation: the jump appears exactly when correction turns on after warmup.

Why Blending Matters

Blended correction

Use **80% corrected update + 20% original update** in the smart configuration.

In the smart configuration, $\beta = 0.8$ keeps 20% of the original client direction.

Why not use full correction all the time?

- Under non-IID data, different clients can have legitimately different update directions.
- Pure correction can over-homogenize those updates toward anchor behavior.
- Blending keeps some client-specific signal while still removing low-bit bias.

Experimental Setup

Data and model

- CIFAR-10
- 10 total clients, full participation
- Shared CNN backbone
- 2 local epochs per round

Baselines

- FedAvg
- FedShift
- Simplified ProWD baseline

Reported regimes

Setting	Bits	Anchors	Rounds
Near-IID	INT8	5/10	50
Near-IID	INT4	5/10	50
Near-IID	INT2	2/10	50
Non-IID	INT8	5/10	50

Question

When does learned dequantization help enough to justify the extra server-side machinery?

FedShift Is the Closest Lightweight Baseline

What FedShift does

- Estimate the round's anchor mean and quantized mean.
- Shift uploads toward the anchor mean before aggregation.

Why it matters

- It adds real correction without any learned model.
- It is stronger than ProWD in most near-IID settings.

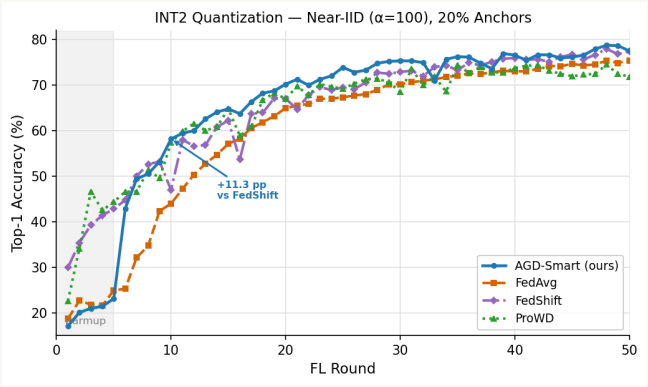
Main limitation

FedShift uses one *global* anchor-driven shift. That works best when clients are near-IID and becomes too crude once client directions diverge.

Why AGD-Smart can go further

- client-specific matching,
- persistent replay instead of round-by-round reset,
- blended correction under non-IID data.

Main Result: INT2 Is Where the Method Helps



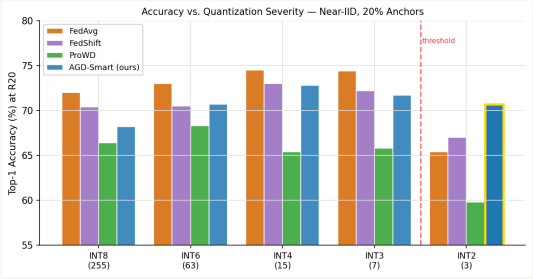
Setting

Near-IID CIFAR-10
INT2 quantization, 20% anchors
50 rounds

FedShift / FedAvg / ProWD

- R10: **58.2** vs 46.9 / 44.0 / 57.3
- R50: **77.5** vs 77.4 / 75.4 / 71.8
- Early edge: **+11.3** over FedShift

Why INT2 Is the Threshold



Key observation

- AGD-Smart only beats FedAvg once quantization becomes too coarse for averaging to recover the signal.

Why INT2 is different

- Only **3 effective levels**: negative, zero, positive.
- Large and small gradients collapse to the same code.
- Opposite signs cancel across clients.

Interpretation

- At INT3 and above, FedAvg usually has enough resolution to survive.
- At INT2, the learned correction becomes genuinely useful.

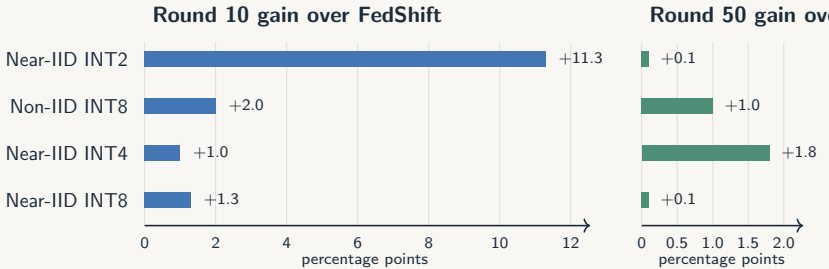
FedShift Comparison at Round 50

Setting	FedAvg	FedShift	ProWD	AGD-Smart
Near-IID, INT2	75.4	77.4	71.8	**77.5**
Near-IID, INT8	**82.7**	81.1	79.3	81.2
Near-IID, INT4	**83.2**	80.7	77.7	82.5
Non-IID, INT8	**80.4**	73.8	72.8	74.8

Interpretation

This view is easier to read because it shows the actual endpoints. FedShift is usually stronger than ProWD, which makes it the more serious correction baseline, but AGD-Smart still recovers the best corrected score in INT2, INT4, and non-IID INT8. FedAvg remains strongest in the easier INT8 and INT4 near-IID settings.

Where AGD-Smart Beats FedShift



- INT2 is the early-convergence story: AGD-Smart is up **+11.3** at round 10 even though the final scores nearly tie.
- INT4 is the clearest end-of-training story: AGD-Smart finishes **+1.8** ahead.
- Non-IID INT8 shows why smart matching matters: a single global shift is too coarse once client distributions diverge.

Other Regimes

Near-IID, INT8

AGD-Smart reaches **81.2** at R50, essentially ties FedShift at 81.1, and still trails FedAvg slightly at the end.

Near-IID, INT4

AGD-Smart reaches **82.5** at R50, beats FedShift by **+1.8**, and finishes within 0.7 of FedAvg.

Non-IID, INT8

AGD-Smart reaches **74.8** at R50, beats FedShift by **+1.0**, but FedAvg still benefits from preserving client diversity.

Across the shared settings, the stable pattern is: **AGD-Smart** $\geq$ **FedShift** $>$ **ProWD** under near-IID data, while **AGD-Smart** $>$ **FedShift** is more meaningful under INT4, INT2 early rounds, and non-IID data.

Takeaways

What the method is doing

It uses clean anchor-client updates to supervise a persistent server-side corrector for low-bit client deltas.

Why it beats ProWD and often FedShift

- replayed supervision instead of round-by-round forgetting,
- client-specific smart matching instead of a single mean target,
- warmup and blending so correction turns on more safely.

Main conclusion

This is not a universal replacement for FedAvg. It is a targeted fix for the regime where quantization is severe enough that averaging alone fails, and it stays most convincing when a simple global shift is not enough, especially INT2 early and non-IID data.

Selected References

- McMahan et al. (2017), *Communication-Efficient Learning of Deep Networks from Decentralized Data*.
- Li et al. (2020), *Federated Optimization in Heterogeneous Networks*.
- Karimireddy et al. (2020), *SCAFFOLD: Stochastic Controlled Averaging for Federated Learning*.
- Wang et al. (2020), *Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization*.
- Reisizadeh et al. (2020), *FedPAQ: A Communication-Efficient Federated Learning Method with Periodic Averaging and Quantization*.
- Yoon et al. (2022), *Bitwidth Heterogeneous Federated Learning with Progressive Weight Dequantization*.
- Seo et al. (2024), *FedShift: Robust Federated Learning Aggregation Scheme in Resource Constrained Environment via Weight Shifting*.
- Chen and Vikalo (2024), *Mixed-Precision Quantization for Federated Learning on Resource-Constrained Heterogeneous Devices*.

Questions?